

## День 2. Алгоритми та обчислювальна складність, частина 2

*Стратегії розроблення алгоритмів та моделі обчислень*

### Стратегія «розділяй та володарюй»

Друга частина алгоритмів присвячена двом темам: загальним стратегіям побудови алгоритмів і теоретичним моделям обчислень. Почнемо зі стратегій — це типові підходи, якими користуються, щоб розв'язати складну задачу.

Перша й найвідоміша стратегія — «розділяй та володарюй», англійською *divide and conquer*. Її ідея в трьох кроках. Перший: розділити задачу на кілька менших підзадач того самого типу. Другий: розв'язати кожну підзадачу, зазвичай рекурсивно, тобто застосувавши до неї той самий метод. Третій: об'єднати розв'язки підзадач у розв'язок початкової задачі.

Класичні приклади ти вже чув у першій частині. Двійковий пошук ділить діапазон навпіл — це розділяй та володарюй. Швидке сортування ділить масив на менші й більші за опорний елемент. Сортування злиттям ділить масив навпіл, сортує половини й зливає їх. Завдяки тому, що задача щоразу зменшується вдвічі, ці алгоритми досягають складності  $O$  від  $n$  на логарифм  $n$  замість квадратичної.

Чому так виходить? Бо глибина поділу — це логарифм від  $n$ , оскільки ми щоразу ділимо навпіл, а на кожному рівні поділу виконується приблизно лінійний обсяг роботи. Логарифм помножити на лінійну роботу дає  $n$  на логарифм  $n$ . Це інтуїція, яку варто розуміти.

## Стратегія балансування

Друга стратегія — балансування. Вона тісно пов'язана з попередньою і означає таке: коли ми ділимо задачу на частини, ці частини варто робити приблизно рівними за розміром. Якщо поділ збалансований, рівні частини, алгоритм працює найефективніше. Якщо ж поділ незбалансований, наприклад одна частина величезна, а інша крихітна, складність погіршується.

Гарний приклад — те саме швидке сортування. Якщо опорний елемент щоразу ділить масив на дві приблизно рівні половини, маємо чудову складність  $n$  на логарифм  $n$ . Але якщо опорний елемент щоразу виявляється найбільшим або найменшим, поділ стає вкрай нерівним, і складність деградує до квадратичної. Тому балансування — це принцип: прагнути рівних частин, щоб тримати складність низькою.

## Динамічне програмування

Третя стратегія — динамічне програмування. Назва трохи оманлива, з програмуванням у звичному сенсі вона мало пов'язана. Суть така: ми розбиваємо задачу на підзадачі, але ці підзадачі перекриваються, тобто повторюються. І замість того щоб розв'язувати ту саму підзадачу багато разів, ми розв'язуємо її один раз, запам'ятовуємо результат у таблиці й потім просто беремо готову відповідь. Це запам'ятовування проміжних результатів називають мемоїзацією.

Класичний приклад — числа Фібоначчі. Якщо обчислювати їх наївною рекурсією, та сама величина рахується знову й знову мільйони разів, і складність стає експоненційною. А якщо запам'ятовувати вже обчислені значення, кожне число рахується лише раз, і складність стає лінійною. Це і є виграш динамічного програмування: ми міняємо зайві обчислення на трохи додаткової пам'яті під таблицю результатів.

Динамічне програмування застосовне там, де є дві ознаки. Перша — оптимальна підструктура: розв'язок задачі будується з розв'язків її підзадач. Друга — перекривні підзадачі: одні й ті самі підзадачі трапляються неодноразово. Типові приклади задач — рюкзак, найкоротші шляхи, порівняння послідовностей.

Підсумуй для себе різницю. «Розділяй та володарюй» ділить задачу на незалежні підзадачі, які не перекриваються. Динамічне програмування працює з підзадачами, що перекриваються, і тому запам'ятовує їхні результати, щоб не рахувати двічі. А балансування — це принцип робити поділ рівним для кращої складності.

## Моделі обчислень

Тепер друга тема — моделі обчислень. Почнемо з двох підходів до програмування. Імперативний підхід описує, як саме досягти результату: це послідовність команд, що крок за кроком змінюють стан програми. Ти ніби даєш покрокову інструкцію виконавцю.

Декларативний підхід описує, що ми хочемо отримати, не вказуючи, як саме це обчислити. Класичний приклад декларативного підходу — мова запитів SQL: ми кажемо, які дані хочемо, а як їх дістати, вирішує система. Запам'ятай коротко: імперативний — це «як», декларативний — це «що».

І нарешті важлива теоретична тема — розв'язність проблем. Не всі задачі взагалі можна розв'язати алгоритмічно, і це треба розуміти. Розрізняють три категорії.

Розв'язна проблема — це проблема, для якої існує алгоритм, що для будь-яких вхідних даних за скінченний час дає правильну відповідь «так» або «ні» і гарантовано зупиняється. Це найкращий випадок.

Напіврозв'язна проблема — це проблема, для якої алгоритм дає відповідь «так», коли відповідь дійсно «так», але у випадку «ні» може працювати нескінченно, так і не давши відповіді. Тобто на позитивних випадках він зупиняється, на негативних — може зависнути.

Нерозв'язна проблема — це проблема, для якої взагалі не існує алгоритму, що розв'язував би її для всіх можливих вхідних даних. Хоч би як ми старалися, універсального алгоритму немає.

Найвідоміший приклад нерозв'язної проблеми — проблема зупинки, англійською halting problem. Її формулювання таке: чи можна написати універсальну програму, яка для будь-якої іншої програми та її вхідних даних визначить, чи ця програма зрештою зупиниться, чи працюватиме нескінченно? Доведено, що такої універсальної програми не існує. Проблема зупинки нерозв'язна. Це фундаментальний результат теорії обчислень, і на іспиті достатньо знати: проблема зупинки — класичний приклад нерозв'язної проблеми.

## Підсумок

Підсумуємо. «Розділяй та володарюй» розбиває задачу на незалежні підзадачі, розв'язує рекурсивно й об'єднує — звідси складність  $n$  на логарифм  $n$ . Балансування вимагає робити поділ на рівні частини для кращої ефективності. Динамічне програмування працює з перекривними підзадачами й запам'ятовує їхні результати, щоб не рахувати двічі. У моделях обчислень імперативний підхід описує «як», декларативний — «що». А проблеми бувають розв'язними, напіврозв'язними та нерозв'язними, причому класичний приклад нерозв'язної — проблема зупинки.

На цьому розділ алгоритмів повністю завершено. Далі за планом — архітектура комп'ютера, операційні системи та мережі.